

Scout — GitHub Epics, Issues & Tech Stack

Full product definition for project management and agent context
From landing page to production deployment

Product Vision

Scout is an AI-powered internship application agent for CS and engineering students. It is not a resume tool. It is not a job board. It is an autonomous agent that reads your resume, optimizes it for every job it finds, and applies on your behalf while you sleep.

The core promise: upload your resume, tell Scout what you want, click "Send Scout", and go back to studying. Scout handles the rest.

Ideal Tech Stack (Final Decisions)

Frontend — packages/web

Next.js 14 with App Router is the frontend framework. It handles all routing, server-side rendering, and the authenticated dashboard. Tailwind CSS handles all styling with shadcn/ui as the component library. Framer Motion handles page transitions. The landing page uses the Ignite Framer template, customized for Scout.

Auth

Clerk handles all authentication — sign up, sign in, session management, and JWT issuance. Clerk's publicMetadata stores onboarding state. Clerk is preferred over Supabase Auth because it handles OAuth, magic links, and session edge cases better out of the box, and its JWT tokens integrate cleanly with both Next.js middleware and FastAPI.

Database

Supabase (PostgreSQL) replaces Convex as the primary database. The reason for this shift is that Scout's job matching requires vector similarity search, and Supabase has pgvector built in — meaning one database handles both relational data (users, resumes, applications) and vector embeddings (resume-to-job matching). This avoids paying for a separate Pinecone instance and keeps the data layer simple.

Supabase also has a file storage layer for resume uploads, row-level security for data isolation, and real-time subscriptions where needed.

AI Backend — packages/api

FastAPI (Python) handles all AI processing, resume parsing, scoring, LaTeX generation, and orchestration of the Scout Agent. Python is non-negotiable here because pdfplumber, python-docx, the Groq SDK, the Gemini SDK, and the Browserbase SDK are all Python-native with the best documentation and support.

AI Models

Groq runs llama-3.3-70b-versatile for quality tasks (resume rewrite, job matching reasoning, copilot chat) and llama-3.1-8b-instant for fast tasks (parsing, scoring, short outputs). Google Gemini gemini-1.5-flash serves as automatic fallback when Groq rate-limits. A single ai_router.py function handles all routing — no AI calls exist outside of it.

Resume Generation

LaTeX via pdflatex running inside a Docker container on Railway. Scout generates a Jake-format .tex file for each optimized resume, compiles it server-side, and returns a PDF. This produces perfectly formatted, ATS-compatible output that no JavaScript PDF library can match. The Docker image includes a full TeX Live distribution.

Vector Search

Supabase pgvector extension handles all vector similarity search. Resume content and job descriptions are both embedded using a lightweight embedding model (either a Groq-supported model or a local sentence-transformers model in FastAPI). Job-to-resume matching uses cosine similarity queries directly in Postgres — no Pinecone needed, no extra cost, no extra service to manage.

Task Queue

Celery with Redis handles all async, long-running, and parallel tasks. This is critical for the Scout Agent — applying to 15 jobs simultaneously means 15 Browserbase browser sessions running in parallel. Celery spawns worker processes that each handle one application independently. Redis serves as both the message broker (task queue) and the result backend (task status). Redis Cloud free tier covers development and early production.

Browser Automation

Browserbase provides cloud browser sessions that the Scout Agent controls programmatically. Each browser session navigates a job portal, fills the application form using the student's parsed resume and the job-specific optimized

resume, and submits. Browserbase handles session management, proxy rotation, and provides a live view of what the agent is doing.

MCP servers provide modular, portal-specific application logic. A Greenhouse MCP server knows exactly how Greenhouse applications work. A Workday MCP server knows Workday's DOM structure. A Lever MCP server knows Lever's form patterns. This keeps the core agent logic clean and makes adding new portals a matter of writing a new MCP server rather than modifying the core system.

Email

Resend handles all transactional email — application completion summaries, follow-up reminders, and account notifications. Resend's free tier covers 100 emails per day which covers early users comfortably.

Payments

Stripe Checkout handles subscription billing. The pricing model is a flat monthly subscription — exact price TBD. Stripe webhooks fire to FastAPI which updates the user's subscription status in Supabase. Pro gates exist in both FastAPI middleware and Next.js route guards.

Deployment

Vercel deploys the Next.js frontend on every push to main. Railway deploys the FastAPI application, the Celery workers, and provides the Redis instance. Each Celery worker is a separate Railway service scaled independently. The LaTeX Docker image lives on Railway alongside FastAPI. Supabase Cloud handles the database.

Monitoring

Sentry tracks errors on both the frontend and FastAPI. Posthog tracks product analytics — resume uploads, scores viewed, jobs selected, Scout Agent runs initiated, applications completed. These two tools give full visibility into both technical failures and user behavior.

Architecture Summary

The request flow for the core Scout Agent run works like this. The user selects jobs on the dashboard and clicks "Send Scout." Next.js calls a FastAPI endpoint which validates their Pro status, fetches their parsed resume and selected jobs from Supabase, and enqueues one Celery task per job into Redis. Each Celery worker picks up a task, spins up a Browserbase session, uses the appropriate MCP server for that portal, fills and submits the application, updates the application status

in Supabase, and exits. When all tasks complete, Resend sends the user a summary email. The dashboard updates in real time via Supabase subscriptions.

GitHub Epics

Each epic below maps to a GitHub Milestone. Each issue under it is a GitHub Issue linked to that milestone. Issues are written to be assigned to a single developer (or a single agent task) and completable in one focused session.

Epic 1: Project Foundation

This epic covers everything that must exist before any feature work begins. It has no user-visible output but every subsequent epic depends on it being solid.

Issue 1.1 — Initialize pnpm monorepo with packages/web and packages/api workspaces, root package.json with shared scripts, and .gitignore covering Node, Python, and environment files.

Issue 1.2 — Initialize Next.js 14 App Router in packages/web with Tailwind CSS, shadcn/ui (slate theme, CSS variables), and next-themes for dark/light mode.

Issue 1.3 — Initialize FastAPI in packages/api with requirements.txt, Dockerfile (includes full TeX Live for LaTeX compilation), and folder structure matching the architecture definition: routes/, services/, core/, prompts/, data/.

Issue 1.4 — Set up GitHub Actions CI workflow: pnpm install, Next.js lint and typecheck, FastAPI pip install and Python syntax check, run on every push and PR.

Issue 1.5 — Create .env.example files for both packages with all required variable names documented and explained, no real values.

Issue 1.6 — Set up Supabase project, create database schema (users, resumes, analyses, jobs, applications, conversations tables), enable pgvector extension, configure row-level security policies for all tables.

Issue 1.7 — Set up Redis Cloud instance (free tier), verify connection from FastAPI, install Celery and configure celery.py with Redis as broker and backend.

Issue 1.8 — Set up Clerk application, create Supabase JWT template in Clerk dashboard, configure Clerk middleware in Next.js protecting all authenticated routes.

Issue 1.9 — Wire Clerk JWT verification into FastAPI as auth middleware so all protected endpoints can identify the requesting user.

Issue 1.10 — Set up Browserbase account, verify API key works, create a test session from FastAPI that opens a browser and navigates to a URL successfully.

Epic 2: Landing Page

The landing page is the first thing every user sees. It must convert visitors into signups. It uses the Ignite Framer template as the visual foundation, customized for Scout's brand and copy.

Issue 2.1 — Set up Ignite Framer template on Framer, customize with Scout branding (name, colors, logo), deploy to a Framer subdomain for review.

Issue 2.2 — Write all landing page copy: headline, subheadline, feature descriptions, social proof section, pricing section, and CTA text. Copy must communicate the core promise clearly: Scout applies to internships for you.

Issue 2.3 — Implement the animated hero section in Next.js (or keep in Framer and embed) with the before/after resume score visualization (42 → 87 animated score rings, red to purple).

Issue 2.4 — Implement the feature highlights section: Resume Intelligence, Role Alignment, Scout Agent (auto-apply). Each with icon, title, and two-sentence description.

Issue 2.5 — Implement the "How Scout works" section: three steps with illustrations (Upload resume → Scout optimizes → Scout applies).

Issue 2.6 — Implement the pricing section with Free and Pro tier comparison. CTA buttons wire to Clerk signup flow.

Issue 2.7 — Implement the social proof section: university logos marquee using Brandfetch for logo fetching, with cache-first lookup, 7-day TTL, and graceful fallback for missing logos.

Issue 2.8 — Add Posthog analytics to landing page: track page views, CTA clicks, and scroll depth milestones.

Issue 2.9 — Ensure landing page is fully responsive at 375px, 768px, and 1280px. Test on both light and dark system preferences.

Epic 3: Auth & Onboarding

This epic covers the complete new-user journey from landing page CTA to first dashboard view. The auth flow must be airtight — broken redirects here kill conversion.

Issue 3.1 — Build Clerk sign-up and sign-in pages using Clerk's prebuilt components, styled to match Scout's design system.

Issue 3.2 — Build the Clerk webhook handler in FastAPI: on user.created event, create a corresponding user record in Supabase with clerkId, email, and default values.

Issue 3.3 — Build the /onboarding page in Next.js: collect target roles (visual selector cards for SWE, ML, Data Eng, DevOps, Product, Research), graduation year, and school name.

Issue 3.4 — Wire onboarding form submission: save to Supabase users table, call Clerk server action to set publicMetadata.onboardingComplete: true, redirect to /dashboard.

Issue 3.5 — Build Next.js middleware redirect logic: unauthenticated users on protected routes go to sign-in, authenticated users without onboarding complete go to /onboarding, authenticated users with onboarding complete pass through.

Issue 3.6 — Build the resume upload step as the first action on /dashboard for new users: drag-and-drop zone, PDF and DOCX only, uploads to Supabase Storage, triggers analysis pipeline automatically on upload.

Epic 4: Resume Intelligence Engine

This is the highest-priority feature engine. Everything else in Scout depends on having a parsed, scored, and optimized resume. Build this before any other engine.

Issue 4.1 — Build core/ai_router.py: single async call_ai function, task parameter selects model (fast=8b, quality=70b), automatic Groq to Gemini fallback on RateLimitError, tenacity retry decorator before fallback triggers.

Issue 4.2 — Build services/resume_parser.py: extract raw text from PDF using pdfplumber, extract from DOCX using python-docx, detect and flag image-only PDFs as unparseable, return clean text string.

Issue 4.3 — Write prompts/parse_prompt.txt: system prompt that extracts structured JSON from raw resume text. Must enforce JSON-only output with no preamble. Schema includes name, email, education array, experience array with bullets, projects array with bullets, and skills object.

Issue 4.4 — Build POST /api/resume/parse endpoint: receives file bytes and MIME type, runs parser service, sends to AI router with parse prompt, returns structured JSON, saves to Supabase resumes table.

Issue 4.5 — Write prompts/score_prompt.txt: scoring rubric prompt for all four dimensions. Experience Quality (0-25), Metrics and Impact (0-25), Structure (0-25), Keywords (0-25). Must return JSON with score, breakdown, and weaknesses array.

Issue 4.6 — Build services/resume_scorer.py: receives parsed resume JSON and target role, constructs scoring prompt with role context, calls AI router with fast model, parses and validates response, returns score object.

Issue 4.7 — Build POST /api/resume/score endpoint: receives parsed resume and target role, returns full score object with breakdown and weaknesses array. Each weakness has type, severity, message, and suggestion fields.

Issue 4.8 — Build POST /api/resume/analyze combined endpoint: runs parse and score in sequence, stores results in Supabase analyses table, returns full result in one response. This is the endpoint the frontend calls after upload.

Issue 4.9 — Build the Scout Score results UI in Next.js: animated score circle (SVG arc animating from 0 to final score), four dimension bars, weaknesses list with severity badges, skeleton loaders while analysis runs.

Issue 4.10 — Add free tier gate: free users see score and weaknesses list. The optimized resume section is blurred with an upgrade CTA overlay for free users.

Epic 5: Resume Optimization Engine

Scout doesn't just score resumes — it rewrites them completely for each job, injecting job-description keywords while maintaining Jake format integrity. This engine has two modes: a general optimization (Jake format cleanup) and a job-specific optimization (keyword injection per JD).

Issue 5.1 — Write prompts/rewrite_prompt.txt: Jake format enforcement prompt. Rules: single column, no tables, no photos, action verb plus task plus result plus metric bullet structure, 1 page for students. Must return complete resume in structured JSON matching the parse schema.

Issue 5.2 — Write prompts/jd_optimize_prompt.txt: job-description-specific optimization prompt. Receives parsed resume JSON plus job description text, returns optimized resume JSON with role-relevant keywords naturally integrated into bullets. Must not fabricate experience — only surface and reframe existing experience.

Issue 5.3 — Build services/resume_rewriter.py: two functions. general_rewrite takes parsed resume and returns Jake-format optimized version. jd_specific_rewrite takes parsed resume plus job description and returns a version optimized for that specific role.

Issue 5.4 — Build POST `/api/resume/rewrite` endpoint: general rewrite for Pro users. Returns optimized resume JSON and a before/after comparison of changed bullets.

Issue 5.5 — Build POST `/api/resume/rewrite-for-job` endpoint: job-specific rewrite. Receives resume ID and job ID from Supabase, fetches both, runs `jd_specific_rewrite`, stores result as a job-specific resume variant in Supabase.

Issue 5.6 — Set up LaTeX pipeline in FastAPI Docker container: verify `pdflatex` is available, build a `compile_latex(tex_string)` function that writes a temp `.tex` file, runs `pdflatex`, returns PDF bytes, cleans up temp files.

Issue 5.7 — Build `services/latex_generator.py`: converts optimized resume JSON into a Jake-format `.tex` string using a LaTeX template. Template must be ATS-safe — no custom fonts, no graphics, standard section commands only.

Issue 5.8 — Build POST `/api/resume/pdf` endpoint: receives resume ID, fetches optimized resume JSON from Supabase, generates LaTeX, compiles to PDF, returns PDF bytes as a file download response.

Issue 5.9 — Build the optimized resume UI in Next.js: side-by-side before/after view, changed bullets highlighted, download PDF button wired to the PDF endpoint.

Epic 6: Job Discovery Engine

Scout finds internships that match the student's resume and goals. Each job card gets a job-specific optimized resume generated for it. This is what makes Scout different from a job board — every result is personalized.

Issue 6.1 — Register for Adzuna Jobs API, build the Adzuna fetching service in FastAPI: GET requests to the jobs search endpoint, filter to internship and entry-level postings in tech roles, normalize response to Scout's internal job schema.

Issue 6.2 — Build the Supabase cron job (using `pg_cron` or a Railway cron service) to refresh job listings daily at 6am UTC. Store results in the jobs table with a 14-day TTL (`expires_at` field). Purge expired listings nightly.

Issue 6.3 — Set up `pgvector` in Supabase: enable the extension, add an embedding column to both the resumes and jobs tables, write a function to generate embeddings for a text string using the embedding model in FastAPI.

Issue 6.4 — Build `services/job_matcher.py`: two scoring layers. Hard score is skills overlap percentage (required skills matched divided by total required). Semantic score is cosine similarity between resume embedding and job description embedding using `pgvector`. Final score combines both with configurable weights.

Issue 6.5 — Build POST /api/jobs/match endpoint: receives user ID, fetches their current parsed resume from Supabase, runs job_matcher against all non-expired listings, returns jobs categorized as STRONG_FIT (>70%), GOOD_FIT (50-70%), and STRETCH (30-50%). Jobs below 30% are excluded.

Issue 6.6 — Build the job card component in Next.js: company logo (Brandfetch), role title, location, remote badge, fit percentage badge, required skills list, and a "View tailored resume" expandable section that shows the JD-specific resume generated for this card.

Issue 6.7 — Build the job discovery page in Next.js: three swimlane columns (Strong Fit, Good Fit, Stretch), job cards with select/deselect checkboxes, filter bar for role type and location, "Send Scout" button fixed at the bottom that activates when at least one job is selected.

Issue 6.8 — Wire job-specific resume generation into the job card: when a student views a job card for the first time, trigger POST /api/resume/rewrite-for-job in the background. Show a generating indicator, then display the tailored resume when ready. Cache result in Supabase so it only generates once per job per user.

Epic 7: Scout Agent (Browser Automation)

This is the crown jewel feature. The Scout Agent applies to jobs autonomously using Browserbase for browser sessions and MCP servers for portal-specific logic.

Issue 7.1 — Build the Celery task definition for a single job application: task receives user ID, job ID, and resume variant ID. Fetches all necessary data from Supabase. Spins up a Browserbase session. Updates application status throughout. Handles errors gracefully and retries up to 2 times on transient failures.

Issue 7.2 — Build the Greenhouse MCP server: understands Greenhouse's application form structure, knows how to fill name, email, resume upload, and custom question fields. Handles Greenhouse's file upload mechanism specifically.

Issue 7.3 — Build the Lever MCP server: understands Lever's application flow, handles their specific form fields and resume parsing behavior. Lever applications are generally simpler than Greenhouse.

Issue 7.4 — Build the Workday MCP server: Workday is the hardest portal. Build a best-effort implementation that handles the most common Workday application flow. Flag applications that fail Workday as requiring manual review rather than retrying indefinitely.

Issue 7.5 — Build the portal detection service: given a job application URL, determine which portal it uses (Greenhouse, Lever, Workday, or unknown) and route

to the appropriate MCP server. Use URL patterns and page content as signals.

Issue 7.6 — Build POST `/api/scout/run` endpoint: receives user ID and array of selected job IDs, validates Pro status, fetches all required data, enqueues one Celery task per job into Redis, returns a run ID for status tracking.

Issue 7.7 — Build GET `/api/scout/status/{run_id}` endpoint: returns the current status of each application task in a Scout run (queued, in_progress, completed, failed). Frontend polls this every 5 seconds to update the live status display.

Issue 7.8 — Build the "Send Scout" confirmation modal in Next.js: shows the list of selected jobs, the resume variant that will be used for each, an estimated time to complete, and a confirm button. After confirm, calls `/api/scout/run` and transitions to the live status view.

Issue 7.9 — Build the live Scout run status UI in Next.js: shows each application as a card with live status updates (Queued → Navigating → Filling form → Submitting → Done). Pulls from `/api/scout/status` every 5 seconds. Shows a completion summary when all tasks finish.

Issue 7.10 — Build CAPTCHA handling strategy: when a Browserbase session encounters a CAPTCHA, pause the task, update status to "needs_attention", send the user an in-app notification and email with a link to manually complete the CAPTCHA via Browserbase's live view URL. Resume the task after the user resolves it.

Epic 8: Notifications & Application Tracking

Scout needs to close the loop with users — tell them what happened, remind them to follow up, and give them a complete record of everything Scout did on their behalf.

Issue 8.1 — Set up Resend account and verify sending domain. Build a `send_email` helper in FastAPI that wraps the Resend API with error handling and logging.

Issue 8.2 — Build the application completion email template: sent when a Scout run finishes. Lists all applications with company name, role title, and status (submitted, failed, needs attention). Includes a link back to the Scout dashboard.

Issue 8.3 — Build the application tracker page in Next.js: kanban board with columns Queued, Applied, Phone Screen, Interview, Offer, Rejected. Cards are created automatically when Scout Agent submits an application. Users can drag cards between columns manually as their status changes.

Issue 8.4 — Build Supabase CRUD functions for the applications table: create (called by Scout Agent on submission), update status (called by user dragging kanban card), and list by user (called by the tracker page).

Issue 8.5 — Build the 7-day follow-up reminder system: a Railway cron job (or Supabase pg_cron) that runs daily, finds applications where applied_at was 7 days ago and follow_up_sent_at is null, generates a personalized follow-up email for each using the 8b model, sends via Resend, updates follow_up_sent_at.

Issue 8.6 — Build POST /api/notifications/followup-template endpoint: generates a personalized follow-up email for a specific application using the student's name, the company, and the role. Returns the email body for the user to review before sending if they prefer.

Epic 9: AI Copilot

The Copilot is the conversational layer that makes Scout feel alive. It knows everything about the student — their resume, their score, their target roles, their application history — and gives specific advice, never generic.

Issue 9.1 — Write prompts/copilot_system.txt: the base system prompt with all context injection placeholders. Must include resume score, breakdown, target roles, readiness status, and a summary of recent application activity. Must explicitly instruct the model never to give generic advice.

Issue 9.2 — Build POST /api/copilot/chat endpoint as a streaming SSE response: receives messages array and user ID, fetches full user context from Supabase, builds the system prompt with injected context, streams Groq response token by token, falls back to Gemini (non-streaming) on rate limit.

Issue 9.3 — Build the /copilot page in Next.js: full-height chat interface, token-by-token streaming display, suggested prompt chips for empty state, conversation history loaded from Supabase on mount.

Issue 9.4 — Wire conversation persistence: save completed conversations to Supabase conversations table after each assistant response. Load full history on page mount. Each message has role, content, and timestamp.

Issue 9.5 — Add free tier gate: free users get 5 copilot messages total (not per session — total lifetime). Show upgrade prompt when limit is reached. Track usage in Supabase users table.

Epic 10: Payments & Monetization

Stripe integration makes Scout a real business. The gate enforcement is the most security-critical code in the product — it must exist in both layers and must be verified before launch.

Issue 10.1 — Create Stripe account, create Scout Pro product with monthly price, copy STRIPE_PRICE_ID, add all Stripe environment variables to both services.

Issue 10.2 — Build POST /api/stripe/create-checkout-session: receives user email, creates Stripe Checkout Session in subscription mode, returns session URL.

Issue 10.3 — Build POST /api/stripe/webhook: verifies Stripe signature, handles customer.subscription.created (set is_pro true in Supabase), customer.subscription.deleted and customer.subscription.paused (set is_pro false). Test all three events with Stripe CLI before moving on.

Issue 10.4 — Build POST /api/stripe/create-portal-session: creates Stripe Billing Portal session for subscription management, returns portal URL.

Issue 10.5 — Build FastAPI require_pro dependency: queries Supabase for user's is_pro status, raises HTTP 403 if false. Apply to all Pro-gated endpoints: resume rewrite, job match, scout run, copilot (after 5 messages).

Issue 10.6 — Build Next.js Pro gate helper in lib/gates.ts: checks user's is_pro from Supabase, redirects to /pricing if false. Apply to all Pro-gated pages.

Issue 10.7 — Build /pricing page in Next.js: Free vs Pro comparison, upgrade CTA wired to checkout session endpoint, designed to convert free users.

Issue 10.8 — Add upgrade CTAs throughout the app at every point where a free user hits a gate: resume rewrite section, job fit scores, Scout Agent run button, copilot message limit. Each CTA is a consistent component, not ad-hoc styling.

Issue 10.9 — Wire post-upgrade experience: after Stripe checkout redirects to /dashboard?upgraded=true, show a success toast and immediately lift all gates (Supabase subscription status should already be updated by webhook).

Epic 11: Polish & Production Launch

This epic runs last. Do not start it until all engines are built and manually tested end to end.

Issue 11.1 — Add Framer Motion page transitions on all route changes using AnimatePresence and consistent enter/exit animations.

Issue 11.2 — Audit every page for missing skeleton loaders on async data. Every Supabase query must show a Skeleton while loading. No blank flashes.

Issue 11.3 — Audit every page for missing empty states. First-time users on every page must see a meaningful empty state with a clear next action, not a blank page.

Issue 11.4 — Full mobile audit at 375px, 768px, and 1280px. Fix all overflow, layout, and touch interaction issues.

Issue 11.5 — Install Sentry on Next.js and FastAPI. Configure alerts for unhandled errors, Stripe webhook failures, and Scout Agent task failures.

Issue 11.6 — Install Posthog. Track: resume_uploaded, score_viewed, rewrite_completed, jobs_loaded, scout_run_initiated, application_submitted, upgrade_clicked, subscription_created.

Issue 11.7 — Run Codex o3 security audit: check IDOR vulnerabilities on all Supabase queries, verify Pro gate exists in both layers for every gated feature, verify Stripe webhook signature check is present, verify CORS is locked to production domain only.

Issue 11.8 — Deploy Supabase to production environment, run all migrations.

Issue 11.9 — Deploy FastAPI and Celery workers to Railway: connect GitHub repo, set all environment variables, verify Dockerfile builds and pdflatex is available, verify Celery workers start and process test tasks.

Issue 11.10 — Set CORS in FastAPI to allow only the production Vercel domain.

Issue 11.11 — Deploy Next.js to Vercel: connect GitHub repo, set all environment variables, verify build passes, verify Clerk and Supabase connections work in production.

Issue 11.12 — Register Stripe webhook endpoint in Stripe Dashboard pointing to the Railway production URL. Verify webhook signature works in production.

Issue 11.13 — Add custom domain in Vercel. Update Stripe success/cancel URLs, CORS settings, and Clerk allowed origins to use the real domain.

Issue 11.14 — Run full end-to-end smoke test: sign up, complete onboarding, upload resume, view score, view optimized resume, download PDF, browse jobs, run Scout Agent on one test listing, verify application logged in tracker, upgrade to Pro via Stripe test mode, verify Pro gates lift.

Issue 11.15 — Ship.

Feature Roadmap (Post-Launch)

These features do not belong in the initial build but should be tracked as future GitHub issues for reference.

Auto-Apply to more portals: iCIMS, Taleo, SmartRecruiters, and direct company career pages without a standardized ATS.

Chrome Extension: a Scout sidebar that activates on any job listing page, shows the student's fit score for that specific role, and lets them queue it for Scout to apply with one click.

Interview Preparation: after Scout applies and a student gets a callback, the Copilot pivots to interview prep mode — generating likely questions based on the JD and the student's resume, running mock interviews, and giving feedback.

Referral Network: connect students who got offers at companies with students currently applying to those companies. Warm referrals dramatically improve interview rates.

University Partnerships: sell Scout to university career centers as a tool they offer their students. B2B revenue on top of the consumer subscription.

This document defines the complete Scout product, tech stack, and GitHub issue structure. Paste it alongside the context document at the start of every agent session. Update it as epics complete and new decisions are made.